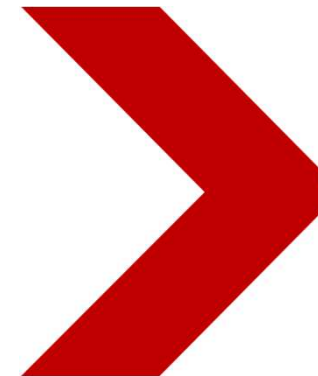


Java程序设计基础

Java Programming Fundamentals

Object class



Object class

- **Object是类层次结构的根类**
 - 每个类都使用Object作为超类
 - 每个类都直接或者间接的继承自Object类
- **如下两种代码表示的含义都是一样的：**
 - `class Person extends Object{}`
 - `class Person{}`



Object类中的主要方法

No.	方法名称	类型	描述
1	<code>public Object()</code>	构造	构造方法
2	<code>public int hashCode()</code>	普通	取得Hash码
3	<code>public final Class<?> getClass()</code>	普通	取得运行时类名称
4	<code>public String toString()</code>	普通	对象打印时调用
5	<code>public boolean equals(Object obj)</code>	普通	对象比较

hashCode()方法

- `public int hashCode()` -- 返回对象的哈希码值。
 - 哈希码值是根据哈希算法计算出来的一个值，这个值和地址值有关（通常通过将对象的内部地址转换而成），但不是实际地址值。

```
public class StudentTest {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        System.out.println(s1.hashCode());  
        Student s2 = new Student();  
        System.out.println(s2.hashCode());  
        Student s3 = s1;  
        System.out.println(s3.hashCode());  
    }  
}
```

getClass()方法

- `public final Class<?> getClass()`
 - 返回此Object的运行时类。
 - Class类表示正在运行的Java应用程序中的类和接口。

```
public class StudentTest {  
    public static void main(String[] args) {  
        Student s1 = new Student();  
        Class c=s1.getClass();  
        String s=c.getName();  
        System.out.println(s);  
        // 链式编程  
        System.out.println(new Student().getClass().getName());  
    }  
}
```

toString()方法

- `public String toString()` 返回对象的字符串表示形式。
 - 此方法返回一个等于下列值的字符串：
 - `getClass().getName() + '@' + Integer.toHexString(hashCode())`
 - `Integer.toHexString(int I)` 把一个整数转换成一个十六进制表示。

```
Student s1=new Student();  
System.out.println(s1.toString());  
System.out.println("-----");  
System.out.println(s1.getClass().getName()+"@"+Integer.toHexString(s1.hashCode()));  
System.out.println("-----");  
System.out.println(s1);
```

当直接输出一个对象的名称时，其实就是调用该对象的toString()方法。

toString()方法

- Object类的toString()方法产生的字符串表示没有意义，建议所有子类覆盖此方法。
- 如何重写：把该类的所有成员变量值组成返回即可。

```
class Student{  
    private int age;  
    private String name;  
    @Override  
    public String toString()  
    {  
        return "年龄:"+age+"姓名:"+name;  
    }  
}
```

注意：可使用集成开发工具自动覆写toString()方法。

equals()方法

- `public boolean equals(Object obj)` 判断其他某个对象是否与此对象 “相等”

```
Student stu1=new Student("张三",19);  
Student stu2=new Student("张三",19);  
System.out.println(stu1.equals(stu2));  
String s1="张三";  
String s2=new String("张三");  
System.out.println(s1.equals(s2));
```

分别查看Object类的equals()和String类的equals()方法源码

Object.equals() vs. String.equals()

```
public boolean equals(Object obj) {  
    return (this == obj);  
}
```

- 默认情况下比较的是地址值。
- String类对equals()进行了重写

JDK1.8 版本实现

```
1 ▼ /**
2  * 比较两个字符串内容是否相等
3  * @param anObject 待比较对象
4  * @return 内容相同返回true, 否则false
5  */
6  @Override
7  public boolean equals(Object anObject) {
8      // 1. 先比较引用地址, 同一个对象直接返回true
9  ▼    if (this == anObject) {
10         return true;
11     }
12     // 2. 判断传入对象是否是String类型, 不是则直接返回false
13 ▼    if (anObject instanceof String) {
14         // 向下转型为String
15         String anotherString = (String)anObject;
16         // 获取底层字符数组长度
17         int n = value.length;
```

```
18         // 3. 长度不同, 内容一定不同, 直接返回false
19 ▼        if (n == anotherString.value.length) {
20             char v1[] = value;
21             char v2[] = anotherString.value;
22             int i = 0;
23             // 4. 逐字符比较, 只要有一个不同就返回false
24 ▼            while (n-- != 0) {
25                 if (v1[i] != v2[i])
26                     return false;
27                 i++;
28             }
29             // 所有字符相同, 返回true
30             return true;
31         }
32     }
33     // 不满足所有条件, 返回false
34     return false;
35 }
```

String.equals()

• JDK 9+ 源码实现

```
1  @Override
2  public boolean equals(Object anObject) {
3      // 引用相同直接返回true
4      if (this == anObject) {
5          return true;
6      }
7      // 模式匹配判断类型+转型
8      return (anObject instanceof String aString)
9          // 检查编码格式一致 (LATIN1/UTF16)
10         && (!COMPACT_STRINGS || this.coder == aString.coder)
11         // 逐字节比较内容
12         && StringLatin1.equals(value, aString.value);
13 }
```

equals()重写示例

```
public boolean equals(Object obj) {  
    if(this==obj) return true; //提高效率  
    if(!(obj instanceof Student)) return false; //提高健壮性  
    Student s=(Student)obj;  
    return this.name.equals(s.name) && this.age==s.age;  
}
```



通过Object类接收任意引用类型的对象

- Object类是所有对象的父类
 - 所有的对象都可以向Object进行转换，其中也包含了数组和接口类型
 - 即：一切的引用数据类型都可以使用Object进行接收。

使用Object接收接口实例

```
interface A{                                     // 定义接口A
    public String getInfo() ;                   // 定义抽象方法
}
class B implements A{                           // 子类实现接口
    public String getInfo(){                   // 覆写接口中的抽象方法
        return "Hello World!!!" ;
    }
}
public class ObjectDemo04 {
    public static void main(String args[]) {
        A a = new B() ;                        // 为接口实例化
        Object obj = a ;                       // 对象向上转型
        A x = (A)obj ;                         // 对象向下转型
        System.out.println(x.getInfo()) ;
    }
}
```

使用Object接收数组

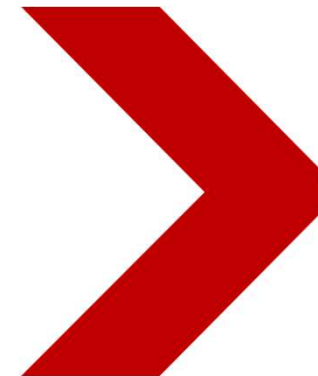
```
public class ObjectDemo05 {  
    public static void main(String args[]) {  
        int temp[] = {1,3,5,7,9} ;  
        Object obj = temp ;  
        print(obj) ;  
    }  
    public static void print(Object o){  
        if(o instanceof int[]){  
            int x[] = (int[])o ;  
            for(int i=0;i<x.length;i++){  
                System.out.print(x[i] + "\t") ;  
            }  
        }  
    }  
}
```

// 定义数组
// 使用Object接收数组
// 传递数组引用
// 接收一个对象
// 判断对象的类型
// 向下转型
// 循环输出

Java程序设计基础

Java Programming Fundamentals

String



实例化String对象

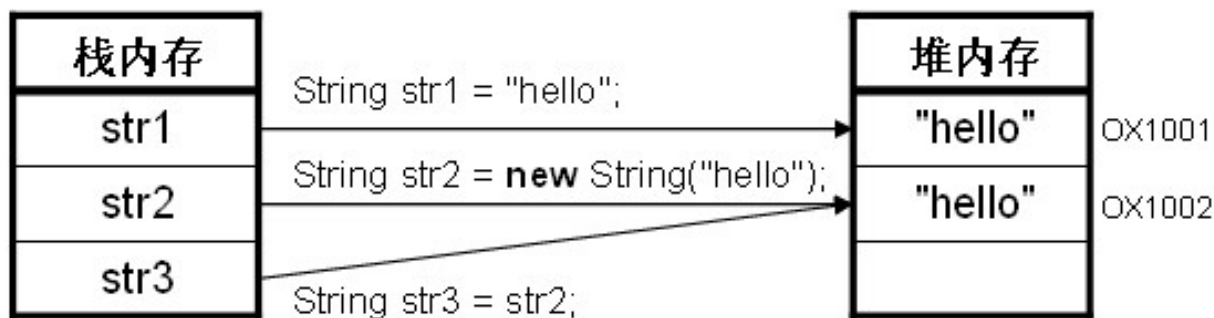
- 直接赋值、通过关键字new

```
public class StringDemo01 {  
    public static void main(String[] args) {  
        String str1 = "国产软件自主可控"; //直接赋值  
        String str2 = new String("从我做起"); //通过关键字new  
        System.out.println(str1);  
    }  
}
```

String的比较方式

- 地址值比较: ==
- 内容比较: equals()方法

```
public class StringDemo04 {  
    public static void main(String[] args) {  
        String str1 = "hello";  
        String str2 = new String("hello");  
        String str3 = str2;  
        System.out.println("str1 == str2 --> " + (str1 == str2));  
        System.out.println("str1 == str3 --> " + (str1 == str3));  
        System.out.println("str2 == str3 --> " + (str2 == str3));  
    }  
}
```



String的比较方式

- 内容比较: equals()方法

```
public class StringDemo05 {  
    public static void main(String[] args) {  
        String str1 = "hello";  
        String str2 = new String("hello");  
        String str3 = str2;  
        System.out.println("str1 equals str2 --> "+(str1.equals(str2)));  
        System.out.println("str1 equals str3 --> "+(str1.equals(str3)));  
        System.out.println("str2 equals str3 --> "+str2.equals(str3));  
    }  
}
```

字符串 – “hello”

- 一个字符串就是String的匿名对象

```
public class StringDemo05 {  
    public static void main(String[] args) {  
        //字符串可以直接调用String类的方法  
        System.out.println("hello".equals("hello"));  
    }  
}
```

在进行字符串内容比较时，避免出现NullPointerException(空指针异常)

String str1=null;

str1.equals("abc") **//错误写法**

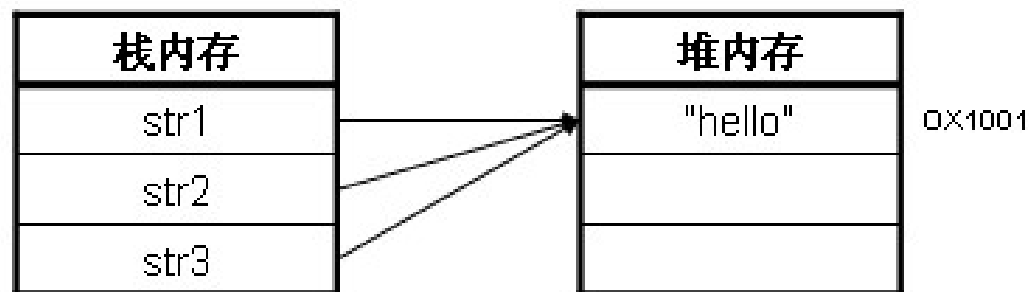
"abc".equals(str1) **//正确写法**

> String类的两种对象实例化方式

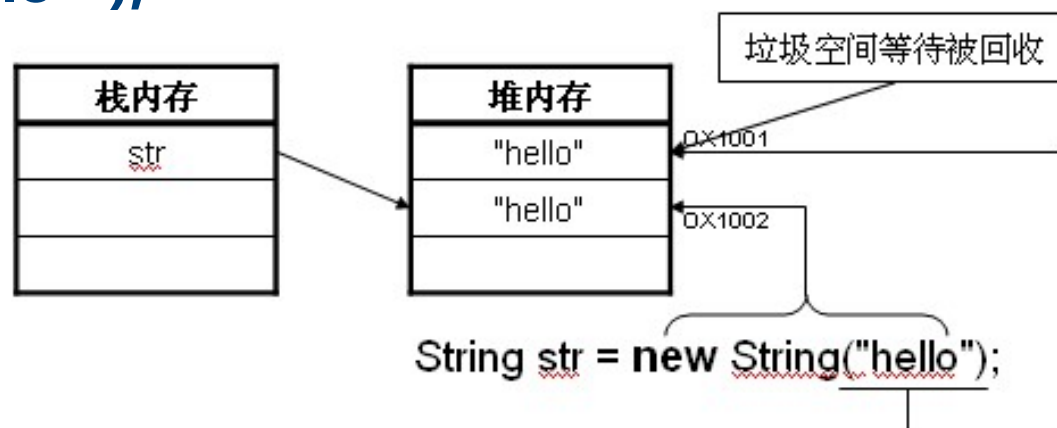
`String str1 = "hello";`

`String str2 = "hello";`

`String str3 = "hello";`



`String str = new String("hello ");`

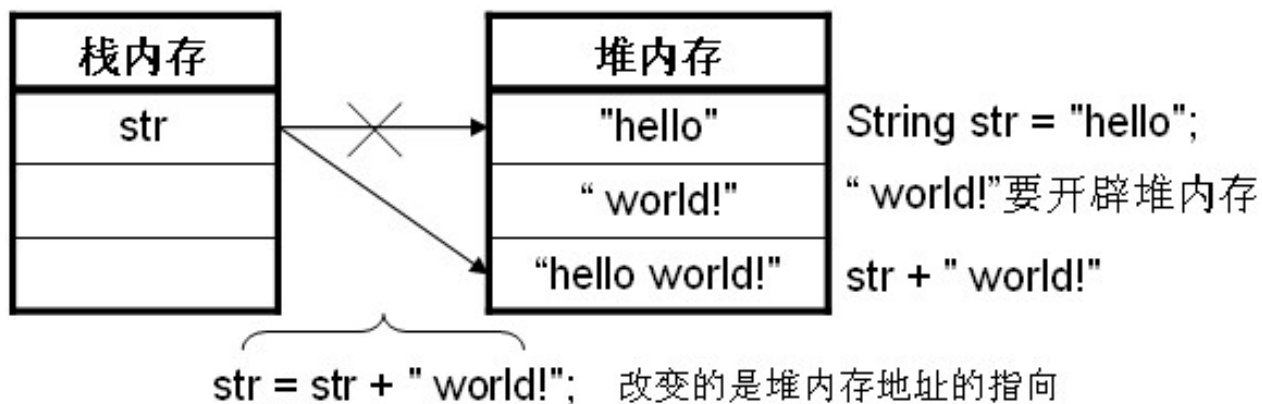


不同实例化方式的区别

- 直接赋值(**String="XXX "**)
 - 只会开辟一块堆内存空间, 并且会自动保存在对象池中供下次重复使用。
- 构造方法(**String=new String("XXX")**)
 - 会开辟两块堆内存空间, 其中有一块将成为垃圾, 并且不会自动保存在对象池。
- **注意:**
 - String对象的实例化都采用直接赋值的方式。

字符串的内容一旦声明则不可改变

```
public class StringDemo09 {  
    public static void main(String[] args) {  
        String str = "hello";  
        str = str + " world!";  
        System.out.println("str = " + str);  
    }  
}
```



String错误应用带来的损耗

```
public class StringDemo10 {  
    public static void main(String[] args) {  
        String str1 = "国产软件自主可控";  
        for (int i = 0; i < 100; i++) {  
            str1 += i;  
        }  
        System.out.println(str1);  
    }  
}
```

- ✓ 在开发中要避免这种情况，上述代码循环修改了100次str1的值，意味着字符串的指向要“断开——连接”100次，并产生很多内存垃圾，性能极低。字符串相连可使用 **StringBuffer** 类完成。

String类的常用方法

No.	方法定义	类型	描述
1	<code>public String(char[] value)</code>	构造	直接将一个字符数组变为一个字符串
2	<code>public String(char[] value,int offset,int count)</code>	构造	将一个指定范围的字符数组变为字符串
3	<code>public String(byte[] bytes)</code>	构造	将一个byte数组全部变为字符串
4	<code>public String(byte[] bytes,int offset,int length)</code>	构造	将指定范围的byte数组变为字符串
5	<code>public char[] toCharArray()</code>	普通	将一个字符串变为字符数组
6	<code>public char charAt(int index)</code>	普通	从一个字符串中取出指定位置的字符
7	<code>public byte[] getBytes()</code>	普通	将一个字符串变为byte数组
8	<code>public int length()</code>	普通	取得字符串长度
9	<code>public int indexOf(String str)</code>	普通	从头开始查找指定的字符串位置
10	<code>public int indexOf(String str,int fromIndex)</code>	普通	从指定位置开始查找指定的字符串位置
11	<code>public String trim()</code>	普通	清除左右两端的空格

String类的常用方法

No.	方法定义	类型	描述
1	<code>public String(char[] value)</code>	构造	直接将一个字符数组变为一个字符串
2	<code>public String(char[] value,int offset,int count)</code>	构造	将一个指定范围的字符数组变为字符串
3	<code>public String(byte[] bytes)</code>	构造	将一个byte数组全部变为字符串
4	<code>public String(byte[] bytes,int offset,int length)</code>	构造	将指定范围的byte数组变为字符串
5	<code>public char[] toCharArray()</code>	普通	将一个字符串变为字符数组
6	<code>public char charAt(int index)</code>	普通	从一个字符串中取出指定位置的字符
7	<code>public byte[] getBytes()</code>	普通	将一个字符串变为byte数组
8	<code>public int length()</code>	普通	取得字符串长度
9	<code>public int indexOf(String str)</code>	普通	从头开始查找指定的字符串位置
10	<code>public int indexOf(String str,int fromIndex)</code>	普通	从指定位置开始查找指定的字符串位置
11	<code>public String trim()</code>	普通	清除左右两端的空格

String类的常用方法

No.	方法定义	类型	描述
12	<code>public String substring(int beginIndex)</code>	普通	从指定位置开始，一直取到尾进行字符串的截取
13	<code>public String substring(int begin,int end)</code>	普通	指定截取字符串的开始点和结束点
14	<code>public String[] split(String regex)</code>	普通	按照指定的字符串对字符串进行拆分 (*)
15	<code>public String toUpperCase()</code>	普通	将一个字符串全部变为大写字母
16	<code>public String toLowerCase()</code>	普通	将一个字符串全部变为小写字母
17	<code>public boolean startsWith(String prefix)</code>	普通	判断是否以指定的字符串开头
18	<code>public boolean endsWith(String suffix)</code>	普通	判断是否以指定的字符串结尾
19	<code>public boolean equals (String str)</code>	普通	判断两个字符串内容是否相等
20	<code>public boolean equalsIgnoreCase(String str)</code>	普通	不区分大小写比较两个字符串是否相等
21	<code>public String replaceAll(String regex,String replacement)</code>	普通	字符串替换 (*)

应用示例-敏感词过滤

- 问题描述

- 在网络程序中，如聊天室、聊天软件等，经常需要对一些用户所提交的聊天内容中的敏感性词语进行过滤。如“性”、“色情”、“爆炸”、“恐怖”、“枪”、“军火”等，这些都不可以在网上进行传播，需要过滤掉或者用其他词语替换掉。

- 解决方案

- 将用户的聊天内容保存到一个字符串对象中，然后与敏感词语类表（数组实现）进行比对。如果属于敏感词语，就过滤掉或替换掉。
- 可使用 `replaceAll(String regex, String replacement)`

应用示例-统计指定词语出现次数

- 问题描述

- 给定一段歌词，从这段歌词中统计出某个词语出现的次数

- 解决方案1:

- `String substring(int beginIndex, int endIndex)` //返回一个新字符串，它是此字符串的一个子字符串。
 - `int indexOf(String str, int fromIndex)` //返回指定子字符串在此字符串中第一次出现处的索引，从指定的索引开始。
 - `boolean contains(CharSequence s)` //当且仅当此字符串包含指定的 char 值序列时，返回 true。

- 解决方案2:

- `String[] split(String regex)` //根据给定正则表达式的匹配拆分此字符串。



应用示例-将给定字符串按字母顺序排序后输出

- 将字符串转换成数组，再对数组进行排序。
 - `char[] toCharArray()`
 - 将此字符串转换为一个新的字符数组。



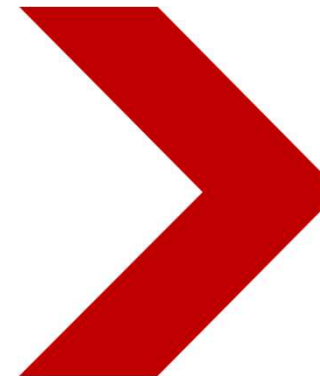
通过Java API学习

- <https://docs.oracle.com/en/java/javase/25/docs/api/index.html>
- 搜索框输入类名 / 方法名, 比如 String、ArrayList、Thread就能看到完整方法说明、参数、返回值、异常等

Java程序设计基础

Java Programming Fundamentals

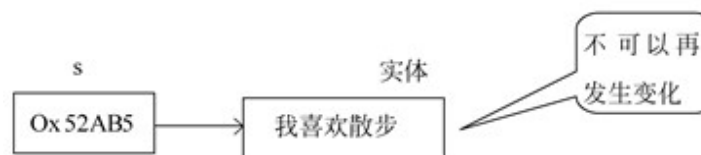
StringBuffer



String $\leftrightarrow$ StringBuffer

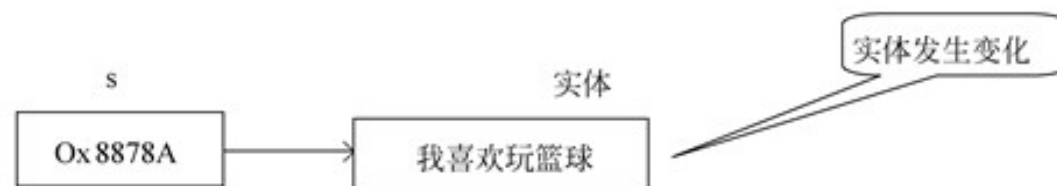
String对象的字符序列是不可修改的，也就是说，String对象的字符序列的字符不能被修改、删除，即String对象的实体是不可以再发生变化的，例如，对于

```
String s = new String("我喜欢散步");
```



StringBuffer类的对象的实体的内存空间可以自动地改变大小，便于存放一个可变的字符序列。比如，对于：

```
StringBuffer s = new StringBuffer("我喜欢");  
对象s可调用append方法追加一个字符序列， s.append("玩篮球");
```



StringBuffer类的常用方法

方法名	参数说明	功能描述
append()	任意数据类型（字符串、数值、字符等）	在字符串末尾追加内容，支持链式调用，最常用的拼接方法
insert()	int offset, 待插入内容	在指定下标位置插入数据，下标从0开始计数
delete()	int start, int end	删除指定区间字符，遵循左闭右开原则
deleteCharAt()	int index	删除指定下标位置的单个字符
replace()	int start, int end, String str	将指定区间字符替换为新字符串
reverse()	无参数	将字符串反转，直接修改原对象
charAt()	int index	获取指定下标位置的字符
setCharAt()	int index, char ch	修改指定下标位置的字符
length()	无参数	获取字符串的有效长度
toString()	无参数	将StringBuffer转为String类型

